



Bloc 3 : Concevoir et mettre en oeuvre des pipelines de données (pour l'IA)

Fraud detection

[Dépôt Github](#)

Contexte et problématique

- **Contexte**

- La fraude : enjeu majeur pour les institutions financières
- En 2019, la BCE estimait les transactions frauduleuses par carte en UE à plus d'1 milliard d'euros
- L'IA permet une détection précise des fraudes → cas d'usage phare en Data Science

- **Problème**

- Les algorithmes de détection sont performants, mais leur déploiement en production reste complexe
- Défi clé : Détecter les fraudes en temps réel et réagir instantanément

Objectifs et périmètre du projet

- **Objectif principal**

- Notification immédiate à chaque détection de fraude : alerte simple
- Rapport quotidien : Vérifier tous les paiements et fraudes de la veille chaque matin

- **Périmètre**

- Cible : Transactions par carte (focus fraude)
- Fonctionnalités
 - Détection en temps réel + système d'alerte
 - Automatisation d'un rapport journalier (paiements + fraudes)

Phase 1 : Entraînement des modèles

Entraînement des modèles - détection de fraude (1)

- **2145 fraudes / ~555k transactions → 0.386%**
- **Fraude : classe ultra-minoritaire → dataset très déséquilibré**
- **Accuracy et ROC-AUC sont trompeurs**
 - même si leurs valeurs peuvent être bonnes, les nombreux vrais négatifs dilue le reste

Donc

- **Utilisation du PR-AUC pour départager les modèles dans un premier temps (pas de dilution avec les négatifs)**
- **Ensuite arbitrage avec f1-score, precision et recall selon le compromis souhaité entre fausses alertes et fraudes manquées**
- **Utilisation méthodes SMOTE ou under-sampling : peu convaincant**

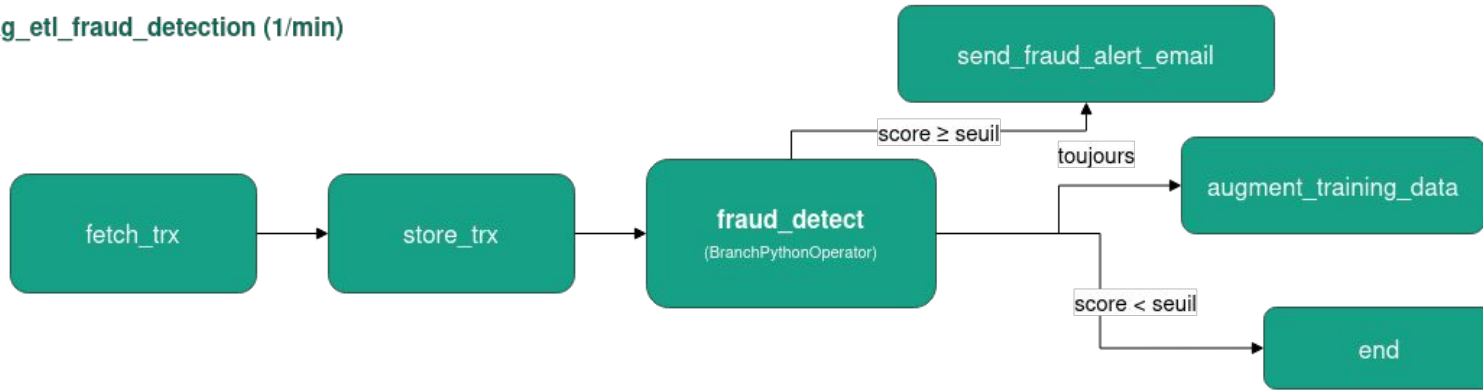
Entraînement des modèles - détection de fraude (2)

Modèle	PR-AUC	F1-score	Précision	Recall
LightGBM (more trees)	0.888	0.80	0.76	0.85
LightGBM (baseline)	0.864	0.77	0.70	0.85
XGBoost	0.844	0.40	0.25	0.96
RandomForest	0.720	0.38	0.24	0.89
LogisticRegression	0.202	0.08	0.04	0.81

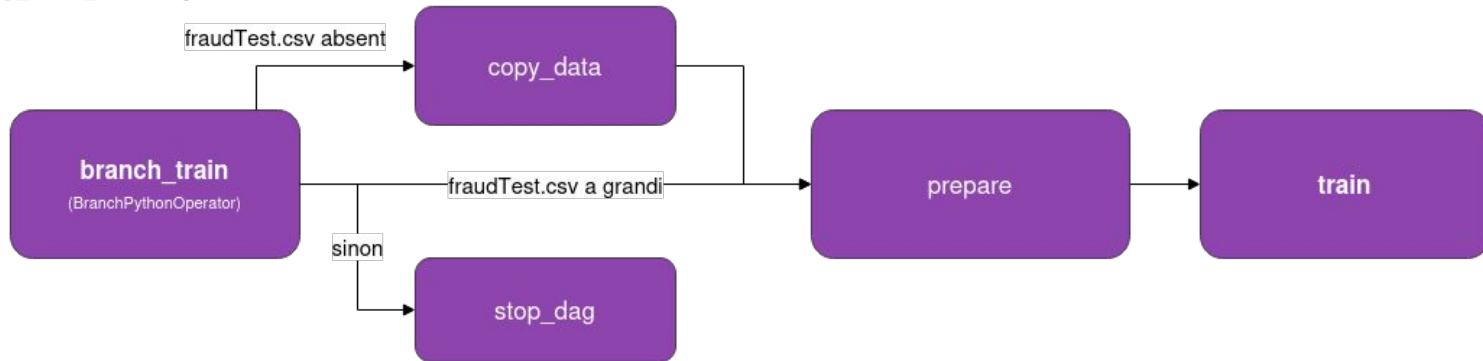
Phase 2 : Mise en place DAGs et Architecture de test

Architecture des DAGs Airflow (dags/dag_*.py)

dag_etl_fraud_detection (1/min)

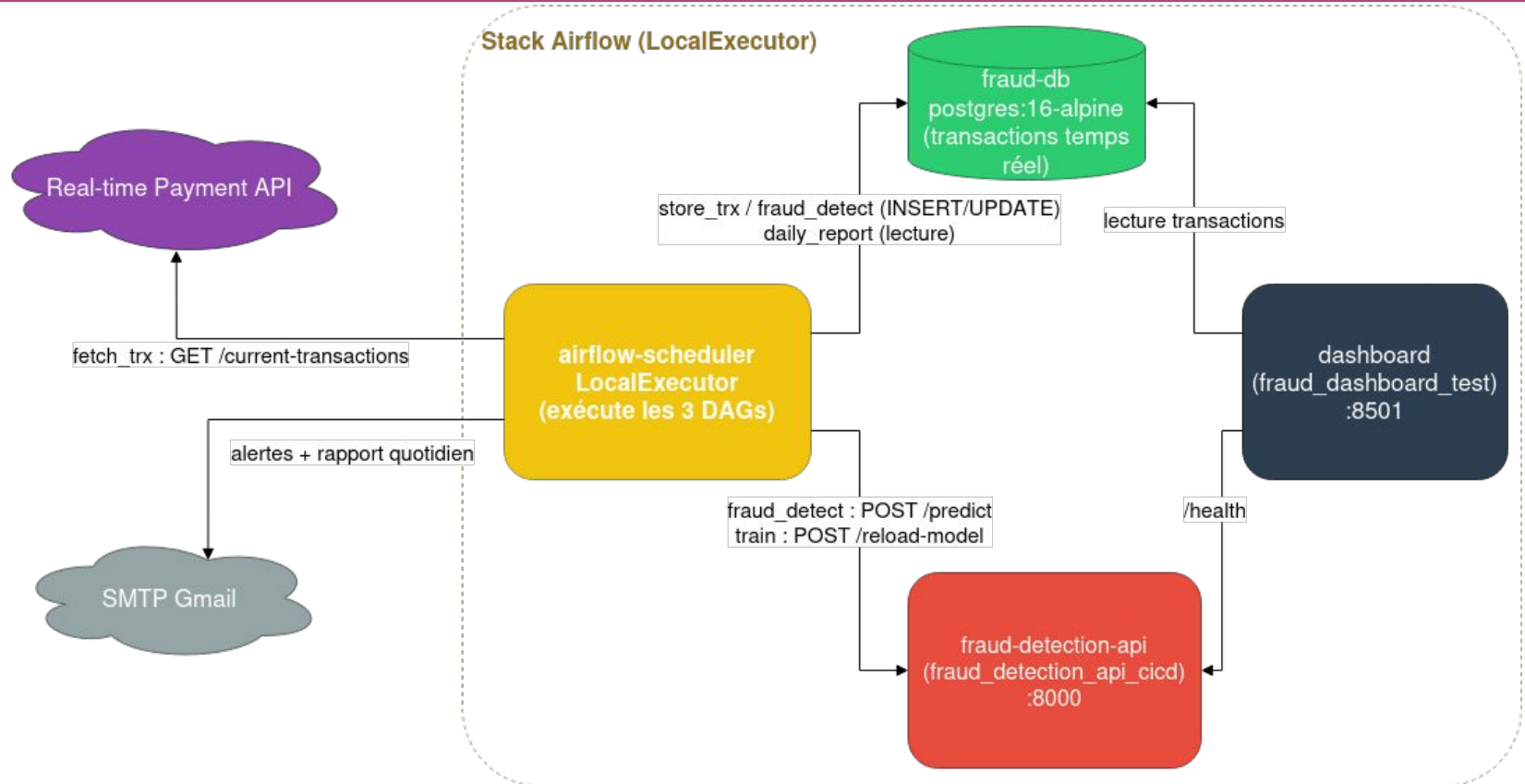


dag_train_model (journalier ou sur seuil)



Architecture de test

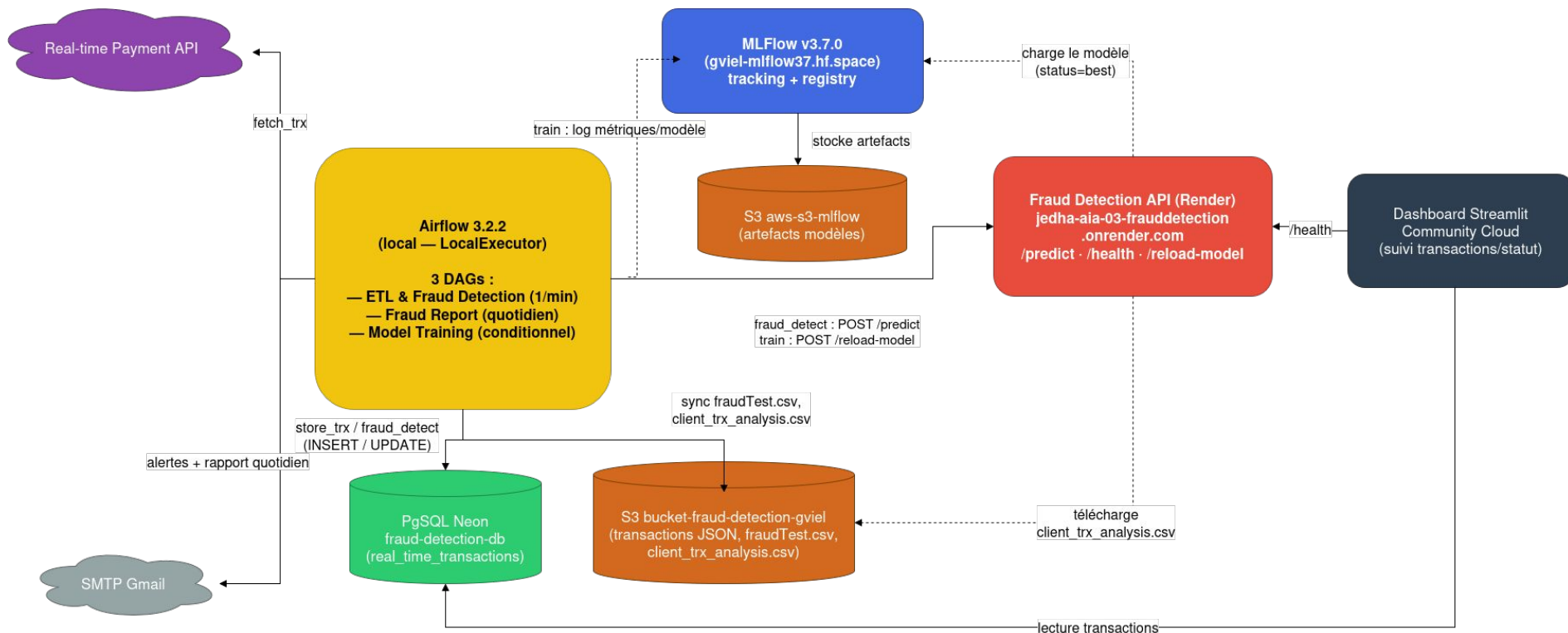
Architecture Test / Intégration — stacks Docker locales



Phase 3 : Mise en Production

Architecture pipeline de production

Architecture Pipeline de Production



Démonstration

- [Dashboard \(Streamlit\)](#)
- [API Fraud Detection \(Render\)](#)
- [Airflow \(stack locale docker compose\)](#)
- [MLFlow \(Hugging Face\)](#)
- [PgSQL DB \(Neon\)](#)
- [Bucket S3 \(AWS\)](#)

Jedha - Data Science Lead

Certification AIA (Architecte IA)



Questions ?